



ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Introduction to Cryptography and Security

Hash Functions

- 1 Introduction
- 2 Security Requirements of Hash Functions
- 3 Generic birthday attack
- 4 The Merkle-Damgard Paradigm

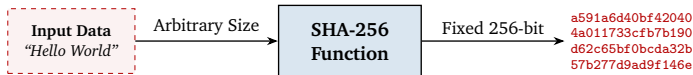
What are Hash Functions?

- Hash functions are an important primitive in cryptography
- Compute a **digest** (or fingerprint) of a message:
 - Short, fixed-length bit string
 - Unique representation of the message
- **Key difference:** Hash functions do not have a key (unlike encryption, MACs)

Applications

- Digital signatures
- Message authentication codes
- Cryptocurrencies
- Pseudo-random number generation
- Password storage
- File verification

SHA-256: A Widely Used Hash Function



Motivation: Digital Signatures

Sign

$$\sigma = \text{Sign}(sk, x) = x^d \bmod n$$

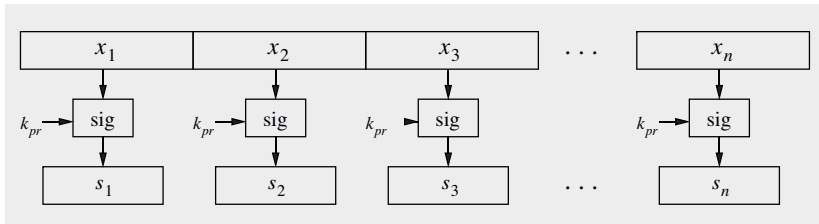
Verify

$$\text{Verify}(pk, x, \sigma) \stackrel{?}{=} \text{accept}$$

Problem

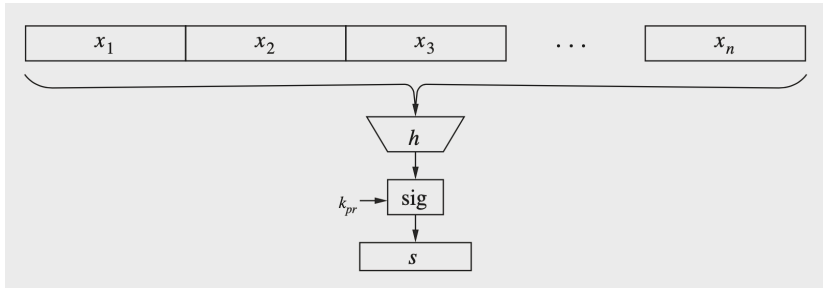
Can only sign short messages $x \in \{0, \dots, n-1\}$; for example, n is 2048 bits

Insecure approach to signing of long messages



- **High computational cost:** requires expensive modular exponentiations for both signer and verifier.
- **Bandwidth overhead:** requires transmitting the full message together with a similarly sized signature.
- **Vulnerable to block manipulation:** an attacker can remove or add individual message blocks along with their corresponding signatures.

Hash-and-Sign Paradigm



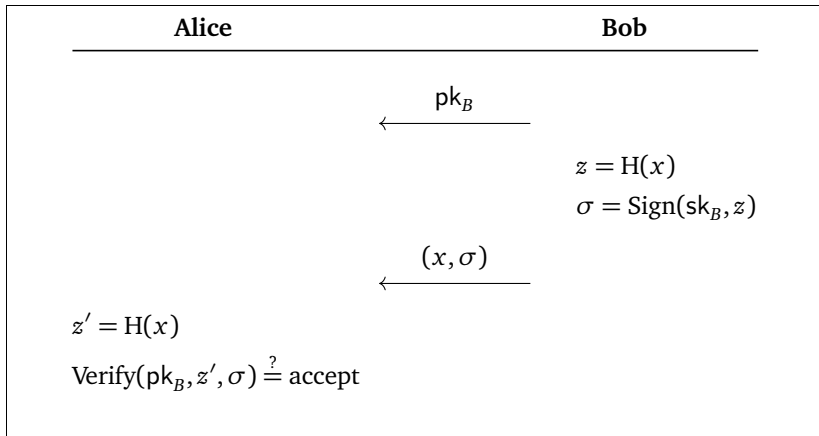
Sign

$$\sigma = \text{Sign}(sk, H(x))$$

Verify

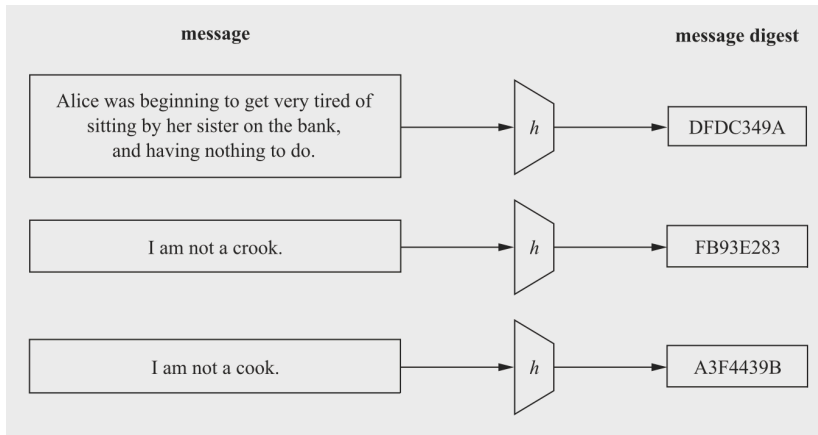
$$\text{Verify}(pk, H(x), \sigma) \stackrel{?}{=} \text{accept}$$

Basic Protocol for Digital Signatures with Hash Function



z is a “fingerprint” of message x , or a **message digest**.

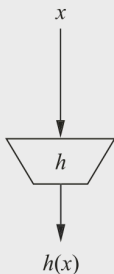
Principal input-output behavior of hash functions



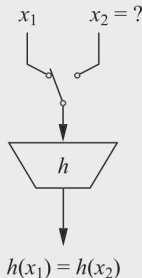
$H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ where n is fixed (e.g., 256 bits for SHA-256).

- 1 Introduction
- 2 Security Requirements of Hash Functions
- 3 Generic birthday attack
- 4 The Merkle-Damgard Paradigm

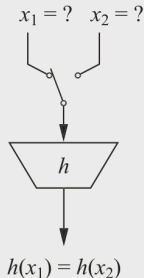
Three fundamental properties



preimage resistance



second preimage resistance



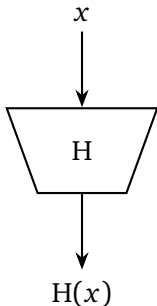
collision resistance

Preimage Resistance or One-Wayness

Think of the hash value z as a **fingerprint**. Given a fingerprint, you cannot reconstruct the original person or message x .

Definition (Preimage Resistance)

Given a hash output z , it must be **computationally infeasible** to find an input message x such that $z = H(x)$.



In digital signatures, a security failure occurs if an attacker finds two **distinct** messages ($x_1 \neq x_2$) with the same hash $H(x_1) = H(x_2)$.

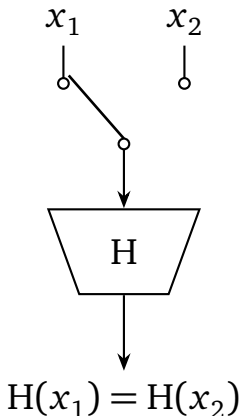
We differentiate between two types:

1. Weak Collision Resistance

- **Given:** A fixed message x_1 .
- **Goal:** Find a different message x_2 such that $H(x_1) = H(x_2)$.

2. Strong Collision Resistance

The attacker is free to choose **both** x_1 and x_2 .



Collision Resistance Attack

Alice

Oscar

Bob

pk_B



x_1



$z = H(x_1)$

$\sigma = \text{Sign}(sk_B, z)$

(x_2, σ)



⚡ substitute

(x_1, σ)



$z' = H(x_2)$

$\text{Verify}(pk_B, z', \sigma) = \mathbf{1}$

Definition (Collision Resistant (C.R.))

We call a hash function **collision resistant** or strong collision resistant if it is **computationally infeasible** to find two different inputs $m_0 \neq m_1$ such that

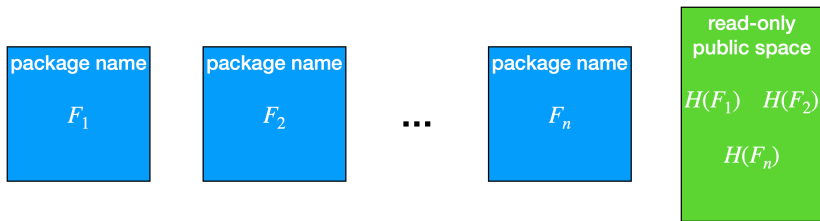
$$H(m_0) = H(m_1).$$

Such a pair (m_0, m_1) is called a **collision**.

Example

SHA-256 (outputs 256 bits)

Protecting file integrity using C.R. hash



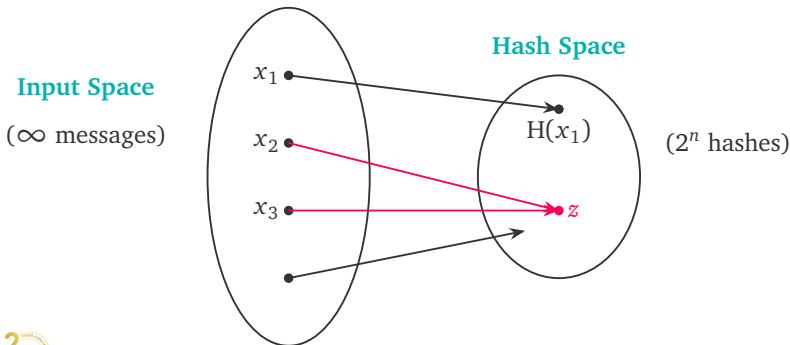
When user downloads package, can verify that contents are valid

- If H collision resistant then attacker cannot modify package without detection
- no key needed (public verifiability), but requires read-only space

- 1 Introduction
- 2 Security Requirements of Hash Functions
- 3 Generic birthday attack
- 4 The Merkle-Damgard Paradigm

Collision inevitability

- **Question:** Can we have hash function without collisions?
- **Answer:** No! Because we have infinitely many messages but only finitely many (2^n) possible hash values (pigeonhole principle). But a good hash function makes finding collisions computationally infeasible.



The Question: How many people are required at a party such that there is a reasonable chance that at least two people have a birthday on the same day?

- **Intuition:** We might assume we need around 183 people (about half of 365 days).
- **Reality:** It turns out we need **far fewer** people.

Let's analyze this using a piecewise probability approach.

Calculating The Probability

We calculate the probability of **no** collision first.

$$\Pr(\text{no collision among 1 people}) = 1$$

$$\Pr(\text{no collision among 2 people}) = 1 \times \left(1 - \frac{1}{365}\right)$$

$$\Pr(\text{no collision among 1 people}) = 1 \times \left(1 - \frac{1}{365}\right) \times \left(1 - \frac{2}{365}\right)$$

Generalizing for t people:

$$\Pr(\text{no collision}) = 1 \times \left(1 - \frac{1}{365}\right) \times \left(1 - \frac{2}{365}\right) \times \cdots \times \left(1 - \frac{t-1}{365}\right)$$

We want the probability of **at least one** collision:

$$\Pr(\text{collision}) = 1 - \Pr(\text{no collision})$$

Surprising Outcome

How many people for a $\approx 50\%$ chance?

- For $t = 23$:

$$1 - \left(1 - \frac{1}{365}\right) \times \cdots \times \left(1 - \frac{22}{365}\right) = 0.507 \approx \mathbf{50\%}$$

- For $t = 40$: The probability rises to $\approx \mathbf{90\%}$.

From Birthdays to Hash Functions

Finding a collision for a hash function $H()$ is exactly the same problem.

Question: How many messages $(x_1, x_2, \dots, x_t)$ must Oscar hash to find a collision $H(x_i) = H(x_j)$ with reasonable probability?

Birthday Party

- Space: 365 days.
- Entities: People (t).
- Collision: Same Birthday.

Hash Function

- Space: 2^n values (n bits).
- Entities: Messages $(x_1, \dots, x_t)$.
- Collision: $H(x_i) = H(x_j)$.

Key Parameter: n is the crucial security parameter.

The Birthday Attack Formula

- Let λ be the collision probability. The number of messages t needed is:

$$t \approx \sqrt{2^{n+1} \ln \left(\frac{1}{1-\lambda} \right)}$$

- Square Root Effect:** For $\lambda \approx 50\%$, this simplifies to:

$$t \approx \sqrt{2^n} = 2^{n/2}$$

(Only need to search the square root of the space)

Security Implication

For a target security level of m **bits**:

Hash Output Length = **2m** bits

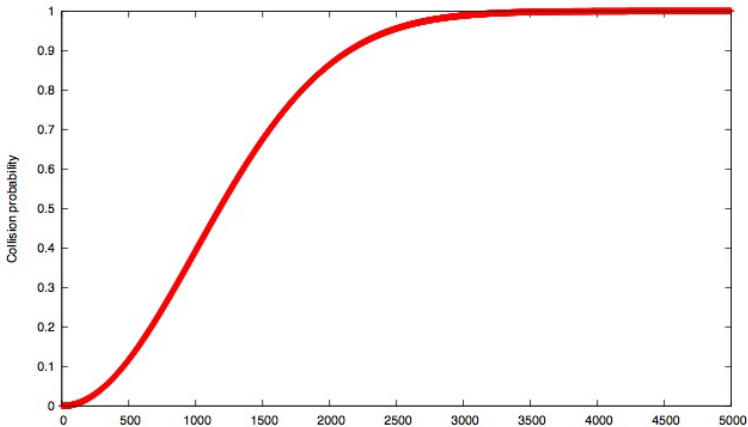
Required Hash Computations

Number of hash computations required (t) for different output lengths (n) and collision probabilities (λ).

n bits	128	160	256	384	512
$\lambda = 0.5$	2^{64}	2^{81}	2^{129}	2^{193}	2^{257}
$\lambda = 0.9$	2^{65}	2^{82}	2^{130}	2^{194}	2^{258}

- Notice that the complexity is roughly $2^{n/2}$.
- Increasing the probability to 0.9 (near certainty) roughly doubles the effort (+1 to the exponent).

Space = 10^6



Let $H : M \rightarrow \{0, 1\}^n$. Collision finding algorithm:

- 1 Choose $2^{n/2}$ random elements in M :

$$m_1, \dots, m_{2^{n/2}}$$

- 2 For $i = 1, \dots, 2^{n/2}$ compute $t_i = H(m_i) \in \{0, 1\}^n$
- 3 Look for a collision ($t_i = t_j$). If not found, got back to step 1.

Expected number of iteration ≈ 2

Running time: $O(2^{n/2})$ (space $O(2^{n/2})$)

Sample C.R. hash functions

Crypto++ 5.6.0 [Wei Dai]

Function	Digest size (bits)	Speed (MB/sec)	Generic attack time
SHA-1	160	153	2^{80}
SHA-256	256	111	2^{128}
SHA-512	512	99	2^{256}
Whirlpool	512	57	2^{256}

Best known collision finder for SHA-1 requires 2^{51} hash evaluations

- 1 Introduction
- 2 Security Requirements of Hash Functions
- 3 Generic birthday attack
- 4 The Merkle-Damgard Paradigm

- Let $H : M \rightarrow T$ be hash function ($|M| \gg |T|$).
- A **collision** for H is a pair $m_0, m_1 \in M$ such that:

$$H(m_0) = H(m_1) \quad \text{and} \quad m_0 \neq m_1$$

- Goal: collision resistant (C.R.) hash functions
- Step 1: given C.R. function for short messages, construct C.R. function for long messages

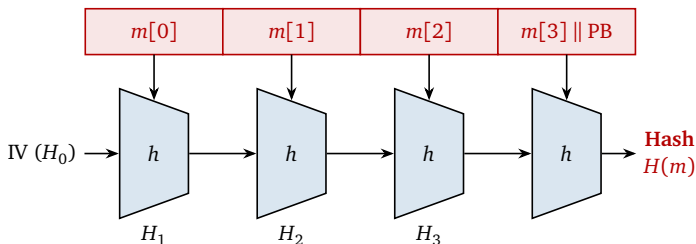
Question

How do we construct collision-resistant hash functions?

Answer

- We start with a collision-resistant function for fixed-size inputs.
- We then build a function that can handle arbitrary-length messages.
- This is done using the Merkle-Damgård construction, which we will explore next.

The Merkle-Damgård iterated construction

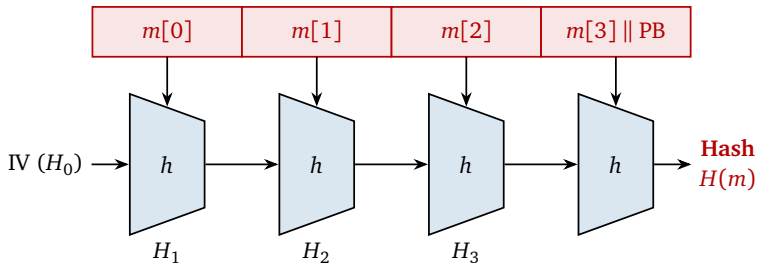


- Given a (compression) function $h : T \times X \rightarrow T$, we obtain $H : X^{\leq L} \rightarrow T$ where H_i are chaining variables and
- PB is padding block

$$1000 \cdots \parallel \overbrace{\text{msg len}}^{64 \text{ bit}}$$

If no space for PB, then add another block.

The Merkle-Damgård iterated construction



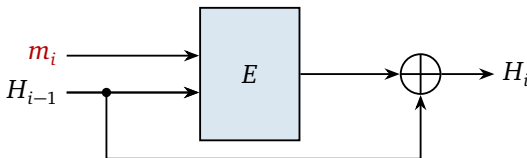
Theorem

If the compression function h is collision resistant, then H is collision resistant.

Compression Functions from Block Cipher

- Let $E : K \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ a block cipher.
- The Davies-Meyer compression function:

$$h(H, m) = E(m, H) \oplus H$$



Suppose we define

$$h(H, m) = E(m, H).$$

Then the resulting $h(., .)$ is not collision resistant.

To build a collision (H, m) and (H', m') choose random (H, m, m') and construct H' as follows:

- ① $H' = D(m', E(m, H))$
- ② $H' = E(m', D(m, H))$
- ③ $H' = E(m', E(m, H))$
- ④ $H' = D(m', D(m, H))$

Other block cipher constructions

Let $E : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ for simplicity.

Miyaguchi-Preneel:

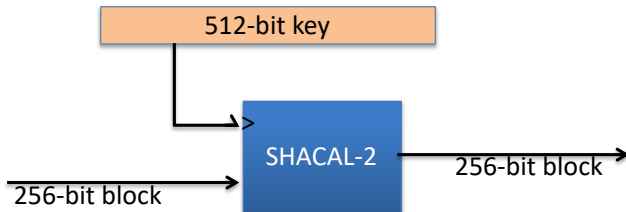
- $h(H, m) = E(m, H) \oplus H \oplus m$ (Whirlpool)
- $h(H, m) = E(H \oplus m, m) \oplus m$

total of 12 variants like this.

Other natural variants are insecure:

$$h(H, m) = E(m, H) \oplus m$$

- Merkle-Damgard function
- Davies-Meyer compression function
- Block cipher: SHACAL-2





25
SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Thank you!



soict.hust.edu.vn/



fb.com/groups/soict

